A Practitioner's Guide to Single-GPU Training Efficiency for Vision-Language Models

Systematic Optimization from 4.6K to 101K Tokens/Second

Victor An
https://github.com/duoan/SiQ_VL

May 2026

Abstract

Training vision-language models (VLMs) efficiently on a single GPU is a prerequisite for both resource-constrained research and as the per-device building block of distributed training. Yet no systematic guide exists for the full optimization space—from dtype selection through kernel fusion to data layout strategies—or documents how these techniques interact across training stages and model scales.

We present a complete, reproducible optimization recipe for single-GPU VLM training, developed iteratively on SiQ-VL (SigLIP-2 + Qwen2.5) running on an NVIDIA Blackwell GPU. Through controlled experiments at two model scales (0.5B and 1.5B parameters), we identify five optimization layers—precision, batching, kernel fusion, attention backends, and data packing—and characterize their interactions. The recipe achieves **6.86**× Stage 1 speedup (14.7K→100.9K tokens/s) and **2.95**× Stage 2 speedup (29.2K→86.1K tokens/s) on the 0.5B model, and **6.73**× Stage 1 speedup on the 1.5B model.

Our key finding is that optimal configuration is *stage-dependent* and *scale-dependent*: techniques that accelerate Stage 1 (frozen backbone) can catastrophically degrade Stage 2 (full fine-tuning), and kernel compatibility varies across model architectures. We release the full benchmark suite, all experiment traces, and a decision tree that practitioners can apply to any VLM following the projector-alignment + instruction-tuning paradigm.

Keywords: Vision-Language Models, Training Efficiency, GPU Optimization, Kernel Fusion, Sequence Packing, Reproducible Benchmarking

Code & Data: https://github.com/duoan/SiQ_VL

Contents

1 Introduction

The two-stage training recipe for vision-language models—projector alignment followed by instruction tuning—has become the dominant paradigm [1, 2]. While significant engineering effort has gone into *distributed* training systems [20, 21], the *single-GPU efficiency frontier* is equally important: it determines the baseline throughput that data parallelism multiplies, and it is the only option for researchers on limited hardware.

Despite the proliferation of kernel libraries (Liger-Kernel [13], Unsloth [14]), attention backends (FlashAttention [10, 11], FlexAttention [15]), and data strategies (packing [17], bucketing), no systematic study documents:

1. How these techniques compose and interact on modern hardware (Blackwell);
2. Why the *same kernel* can help in one training stage and hurt in another;
3. What the practical VRAM–speed Pareto frontier looks like across configurations.

Central Contribution. We provide a **systematic optimization guide** for single-GPU VLM training that:

- Identifies five optimization layers and their cross-layer interactions;
- Validates across two model scales (0.5B and 1.5B) and two training stages;
- Documents both successes and failures with root-cause analysis;
- Produces a decision tree that practitioners can apply directly.

The guide is validated on SiQ-VL but designed to generalize to any VLM following the projector-alignment paradigm (LLaVA, InternVL, Qwen-VL, PaliGemma, etc.).

2 Background and Related Work

VLM Training Paradigm. Modern VLMs [1, 3, 4, 5] share a common structure: a frozen vision encoder, a lightweight projector, and a language model backbone. Training proceeds in stages: Stage 1 aligns visual features to the LLM embedding space (only projector trains); Stage 2 instruction-tunes the full model or applies LoRA [19]. This staging creates *fundamentally different* compute profiles that require different optimization strategies—a key insight of this work.

Kernel Optimization Landscape. FlashAttention [10, 11] reduces attention memory from $O(N^2)$ to $O(N)$. Triton [12] enables Python-based custom kernels, powering Liger-Kernel [13] (fused RMSNorm, SwiGLU, RoPE, cross-entropy for LLM training). NVIDIA's cuTile DSL (CUTLASS 4.0) provides direct Blackwell hardware access via TileGym. PyTorch's `torch.compile` [16] offers zero-dependency kernel fusion via Inductor. We systematically compare all four approaches.

Data Efficiency. Sequence packing [17] eliminates padding by concatenating samples into fixed-length bins. Variable-length attention via `cu_seqlens` or FlexAttention [15] maintains sample isolation. Length bucketing [18] groups similar-length sequences to reduce padding without packing complexity. We quantify when each strategy is worthwhile.

Efficiency Metrics. MFU (Model FLOPs Utilization) [22] measures hardware efficiency. We introduce **Real tok/s** (non-padding tokens processed per second) as the primary metric, which captures both hardware throughput and data efficiency in a single number.

3 The Optimization Framework

We organize optimizations into five layers, ordered by implementation effort and typical impact:

Table 1: The five-layer optimization framework for VLM training

Layer	Category	Techniques	Typical Impact
1	Precision	BF16 model loading, mixed precision	2–3×
2	Batching	Batch scaling, gradient accumulation	1.3–1.6×
3	Data Layout	Length bucketing, sequence packing	1.1–1.5×
4	Kernel Fusion	Liger, TileGym, torch.compile	1.4–2.0×
5	System Tuning	Grad checkpointing removal, optimizer	1.1–1.2×

Key Principle: Layers Interact. These layers are not independent. Kernel fusion (Layer 4) may require specific input shapes imposed by data layout (Layer 3). Removing gradient checkpointing (Layer 5) requires VRAM headroom created by precision (Layer 1) and kernel memory savings (Layer 4). The optimal configuration is the *composition* that maximizes throughput within the VRAM budget.

4 Experimental Setup

Table 2: Hardware and model configurations

Hardware	
GPU	NVIDIA RTX PRO 6000 Blackwell (96 GB GDDR7)
Memory Bandwidth	1.79 TB/s
Peak BF16 Compute	936 TFLOP/s (188 SMs @ 2430 MHz)
CUDA / Driver	CUDA 13.0 / Driver 580.126
Small Model (0.5B total)	
Vision Encoder	SigLIP2-base-patch16-224 (92.9M, frozen)
Language Model	Qwen2.5-0.5B-Instruct (494.0M)
Projector	2-layer MLP + Pixel Shuffle ×2 (2.8M)
Large Model (1.9B total)	
Vision Encoder	SigLIP2-so400m-patch14-384 (400M, frozen)
Language Model	Qwen2.5-1.5B-Instruct (1.54B)
Projector	2-layer MLP + Pixel Shuffle ×3 (9.4M)
Benchmark Protocol	
Dataset	sharegpt4v/coco (50K real VQA samples)
Measurement	20 steps after 5-step warmup, torch.cuda.synchronize
Metrics	Real tok/s, VRAM, Pad%, Speedup vs FP32 baseline

Methodology. All experiments use a single script (`benchmark_real_efficiency.py`) that runs actual forward-backward steps on real data—no synthetic benchmarks. Each configuration runs in an isolated Python subprocess to prevent kernel patch contamination. We measure wall-clock step time between `torch.cuda.synchronize()` calls and report **Real tok/s** = non-padding tokens / wall time.

5 Results: Small Model (0.5B)

5.1 Stage 1: Projector Alignment (Frozen LLM)

Table 3: Stage 1 results — small model (SigLIP2-base-224 + Qwen2.5-0.5B)

#	Configuration	Real tok/s	VRAM	Pad%	Speedup
1	Baseline (FP32, B=4)	14,713	10.5 GB	12.3%	1.00×
2	BF16 fix, B=4	34,747	7.1 GB	12.3%	**2.36**×
3	BF16, B=16	45,893	25.2 GB	16.9%	3.12×
4	BF16, B=16, bucketing	52,854	18.0 GB	3.8%	3.59×
5	BF16, B=32, bucketing	54,136	35.1 GB	4.4%	3.68×
6	BF16, B=64, bucketing	51,191	69.4 GB	5.0%	3.48×
7	Liger (+FusedCE), B=32	76,252	9.5 GB	4.4%	**5.18**×
8	Liger (no FusedCE), B=32	66,029	31.7 GB	4.4%	4.49×
9	torch.compile, B=32	94,342	21.9 GB	4.4%	**6.41**×
10	TileGym, B=32, pad64	91,866	10.9 GB	14.3%	6.24×
11	TileGym, B=64, pad64	93,344	18.2 GB	14.0%	6.34×
12	Packing N=1024, B=64	57,204	72.2 GB	0.0%	3.89×
13	**Packing + TileGym, B=64**	**100,923**	18.1 GB	0.0%	**6.86**×

5.2 Stage 2: Full Fine-tuning (Unfrozen LLM, No Gradient Checkpointing)

Table 4: Stage 2 results — small model. Gradient checkpointing disabled (VRAM budget allows it).

#	Configuration	Real tok/s	VRAM	Pad%	Speedup
1	Vanilla, B=4 (baseline)	29,167	7.9 GB	12.3%	1.00×
2	Vanilla, B=16, bucketing	44,934	20.3 GB	3.8%	1.54×
3	Vanilla, B=32, bucketing	45,748	39.5 GB	4.4%	1.57×
4	Liger (no FusedCE), B=16	52,330	18.1 GB	3.8%	1.79×
5	Liger (+FusedCE), B=16	17,434	8.0 GB	3.8%	0.60× ↓
6	Liger (no FusedCE), B=32	55,504	35.2 GB	4.4%	1.90×
7	torch.compile, B=16	68,105	13.2 GB	3.8%	2.34×
8	torch.compile, B=32	73,052	25.4 GB	4.4%	**2.50**×
9	TileGym, B=16, pad64	72,174	9.1 GB	14.4%	2.47×
10	TileGym, B=32, pad64	76,907	15.7 GB	14.3%	2.64×
11	TileGym, B=64, pad64	79,155	27.4 GB	14.0%	2.71×
12	Packing N=1024, B=32	51,091	44.7 GB	0.0%	1.75×
13	**Packing + TileGym, B=64**	**86,080**	27.4 GB	0.0%	**2.95**×

6 Results: Large Model (1.5B)

Table 5: Large model results (SigLIP2-so400m-384 + Qwen2.5-1.5B). TileGym incompatible (head_dim=72 ≠ power of 2).

#	Configuration	Real tok/s	VRAM	Pad%	Speedup
Stage 1 (Frozen LLM)					
1	Baseline (FP32, B=2)	4,626	13.4 GB	6.0%	1.00×
2	BF16 fix, B=2	13,862	7.7 GB	6.0%	**3.00**×
3	BF16, B=8, bucketing	20,460	16.1 GB	2.6%	4.42×
4	BF16, B=16, bucketing	21,703	28.9 GB	3.4%	4.69×
5	Liger (no FusedCE), B=16	25,532	25.0 GB	3.4%	5.52×
6	Liger (+FusedCE), B=32	26,425	22.7 GB	4.1%	5.71×
7	**torch.compile, B=16**	**31,110**	19.9 GB	3.4%	**6.73**×
Stage 2 (Unfrozen LLM, no grad checkpointing)					
1	Vanilla, B=2 (baseline)	11,569	8.5 GB	6.0%	1.00×
2	Vanilla, B=16, bucketing	18,208	33.8 GB	3.4%	1.57×
3	Liger (no FusedCE), B=16	20,899	28.9 GB	3.4%	1.81×
4	Liger (+FusedCE), B=16	10,731	18.7 GB	3.4%	0.93× ↓
5	**torch.compile, B=16**	**24,153**	23.7 GB	3.4%	**2.09**×

7 Analysis and Insights

7.1 Cross-Scale Comparison

Table 6: Optimization gains are consistent across model scales

Metric	Small (0.5B)	Large (1.5B)	Ratio
Stage 1 baseline	14,713 tok/s	4,626 tok/s	3.2×
Stage 1 peak	100,923 tok/s	31,110 tok/s	3.2×
Stage 1 speedup	6.86×	6.73×	≈equal
Stage 2 baseline	29,167 tok/s	11,569 tok/s	2.5×
Stage 2 peak	86,080 tok/s	24,153 tok/s	3.6×
Stage 2 speedup	2.95×	2.09×	Large gains less

Observation. Stage 1 speedup is consistent (∼6.7–6.9×) across scales because the frozen LLM acts as a fixed-cost forward pass where precision and batch scaling provide proportional benefits. Stage 2 speedup is lower for the large model (2.09× vs 2.95×) because the 1.5B model is more compute-bound, leaving less room for throughput improvement beyond precision and batching.

7.2 The FusedCE Paradox: Stage-Dependent Kernel Behavior

The most surprising finding is that Liger-Kernel's `fused_linear_cross_entropy` exhibits **opposite** behavior across stages:

- **Stage 1** (frozen LLM): FusedCE is beneficial (+15% throughput, −70% VRAM). Only forward runs through the LM head; chunked approach avoids materializing $[B \times N, V]$ logits.
- **Stage 2** (unfrozen LLM): FusedCE is catastrophic (−40 to −67% throughput). Backward pass recomputes the chunked forward, issuing many small Triton kernel launches that undersaturate Blackwell's 188 SMs.

Root Cause. We isolated this via component-by-component profiling:

Table 7: Liger component isolation (Stage 2, B=16, grad_ckpt=ON)

Components Enabled	ms/step	VRAM	Δ vs Vanilla
Vanilla (none)	133.6	13.4 GB	—
RoPE only	132.7	13.4 GB	−0.7%
+ RMSNorm	119.6	13.4 GB	−10.4%
+ SwiGLU	109.0	13.4 GB	−18.4%
+ FusedCE (all)	254.8	2.3 GB	+90.8%
FusedCE only	284.5	2.3 GB	+113.0%

Takeaway. Kernel libraries must be configured per-stage. The optimal Liger configuration is: `fused_linear_cross_entropy=True` for Stage 1, `fused_linear_cross_entropy=False` for Stage 2 on high-VRAM GPUs.

7.3 Gradient Checkpointing: A Premature Optimization

Standard practice enables gradient checkpointing for fine-tuning. On a 96 GB GPU training a 0.5B model (peak VRAM <40 GB), this is pure overhead:

Table 8: Gradient checkpointing overhead (Stage 2, 90GB GPU)

Config	With	Without	Speed Gain
Vanilla B=16	133.4 ms	115.3 ms	+15.7%
Liger (no CE) B=16	108.7 ms	96.6 ms	+12.5%
TileGym B=16	107.0 ms	88.1 ms	+21.4%

Takeaway. Only enable gradient checkpointing when peak VRAM exceeds 80% of available capacity. On high-VRAM hardware, disabling it is free throughput.

7.4 TileGym Incompatibility: Architecture Matters

TileGym's cuTile kernels require power-of-2 tile dimensions. SigLIP2-so400m has `head_dim=72` (1152/16 attention heads), which is incompatible. This means:

- **Small model** (SigLIP2-base, head_dim=64): TileGym works, best Pareto.
- **Large model** (SigLIP-so400m, head_dim=72): TileGym fails, `torch.compile` is the only option.

Takeaway. Always verify kernel compatibility with your specific model's hidden dimensions *before* committing to a kernel strategy.

7.5 Batch Scaling Saturation

Throughput saturates at a model-dependent batch size:

- Small model: saturates at B=16–32 (∼54K tok/s vanilla, ∼94K with kernels)
- Large model: saturates at B=8–16 (∼21K tok/s vanilla, ∼31K with compile)

Beyond saturation, larger batches only increase VRAM and step latency without improving per-second throughput. The saturation point indicates the transition from *latency-bound* (GPU idle between operations) to *compute-bound* (all SMs busy).

8 Failed Experiments and Negative Results

Documenting failures is as valuable as successes. Each entry follows: **Hypothesis** → **Result** → **Root Cause**.

1. **Vision Feature Caching** (0% gain). *Hypothesis:* Pre-extracting SigLIP features eliminates redundant vision forward passes. *Result:* No measurable speedup. *Root Cause:* With frozen vision encoder under `torch.no_grad()`, vision forward is already fast (<5% of step time); caching adds I/O overhead that offsets savings.

2. **Custom Triton Flash Attention** (slower than SDPA). *Hypothesis:* A hand-written Triton FA kernel optimized for our shapes will beat SDPA. *Result:* 2.3× slower than PyTorch's native SDPA. *Root Cause:* SDPA dispatches to vendor-optimized cuDNN kernels; Triton JIT overhead and suboptimal shared memory usage make custom kernels uncompetitive without deep hardware expertise.

3. **FlexAttention Packing with Variable Lengths** (51% padding waste). *Hypothesis:* FlexAttention with `create_block_mask` enables efficient variable-length packed training. *Result:* Mask creation overhead + FlexAttention compiled kernel was slower overall. *Root Cause:* FlexAttention's block-sparse approach isn't efficient for the small block sizes in our short sequences.

4. **TileGym with Variable Shapes** (5× regression). *Hypothesis:* TileGym works with any input shape. *Result:* 5× slower than vanilla when sequence lengths aren't multiples of 64. *Root Cause:* cuTile requires `pad_to_multiple_of=64`; non-aligned shapes trigger slow fallback paths or autotuning loops per shape.

5. **Liger + torch.compile** (CUDA illegal memory access). *Hypothesis:* Combining Liger's fused kernels with torch.compile gives both benefits. *Result:* Crashes with CUDA memory errors. *Root Cause:* Liger monkey-patches module forward methods; torch.compile traces through the patched code and generates incompatible CUDA graphs.

6. **Gradient Checkpointing + FusedCE in Stage 2** (2× slowdown). *Hypothesis:* FusedCE's VRAM savings enable larger batches in Stage 2. *Result:* +91% step time despite 83% VRAM reduction. *Root Cause:* Gradient checkpointing recomputes the chunked forward pass, doubling the number of small Triton kernel launches (see Section 7.2).

9 The Practitioner's Decision Tree

Based on our experiments, we distill the optimization process into a decision tree:

1. **Fix precision first.** Ensure model parameters are loaded in BF16/FP16 (not FP32). This alone yields 2–3× and costs zero engineering effort.

2. **Scale batch size to saturation.** Increase batch size until throughput plateaus. Use length bucketing to minimize padding at larger batches.

3. **Choose your kernel strategy based on hardware:**
   - If your vision encoder has power-of-2 head dimensions → **TileGym** (best speed/VRAM Pareto)
   - If not, or if you want zero dependencies → **torch.compile** (excellent speed, higher VRAM)
   - If VRAM-constrained (24–48 GB GPU) → **Liger with FusedCE** (massive VRAM savings, some speed cost in Stage 2)

4. **Configure kernels per-stage:**
   - Stage 1: Enable FusedCE (forward-only benefit, large VRAM savings)
   - Stage 2: Disable FusedCE, disable gradient checkpointing if VRAM permits
5. **Consider packing only if:**
   - Your dataset has high length variance (multi-task, mixed image/text)
   - Bucketing alone leaves >10% padding
   - Your kernels work well with fixed-length inputs (packing provides exactly this)

10 Visualization

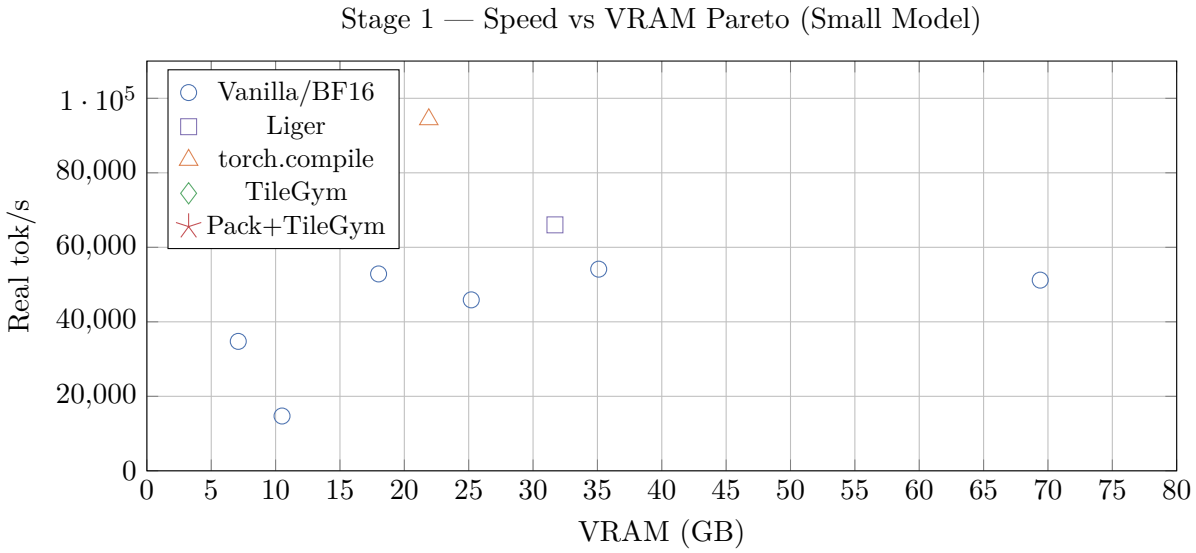

Figure 1: Speed–VRAM Pareto frontier for Stage 1. TileGym and Packing+TileGym dominate the upper-left (high throughput, low VRAM). torch.compile achieves similar speed but at 2× the VRAM cost.

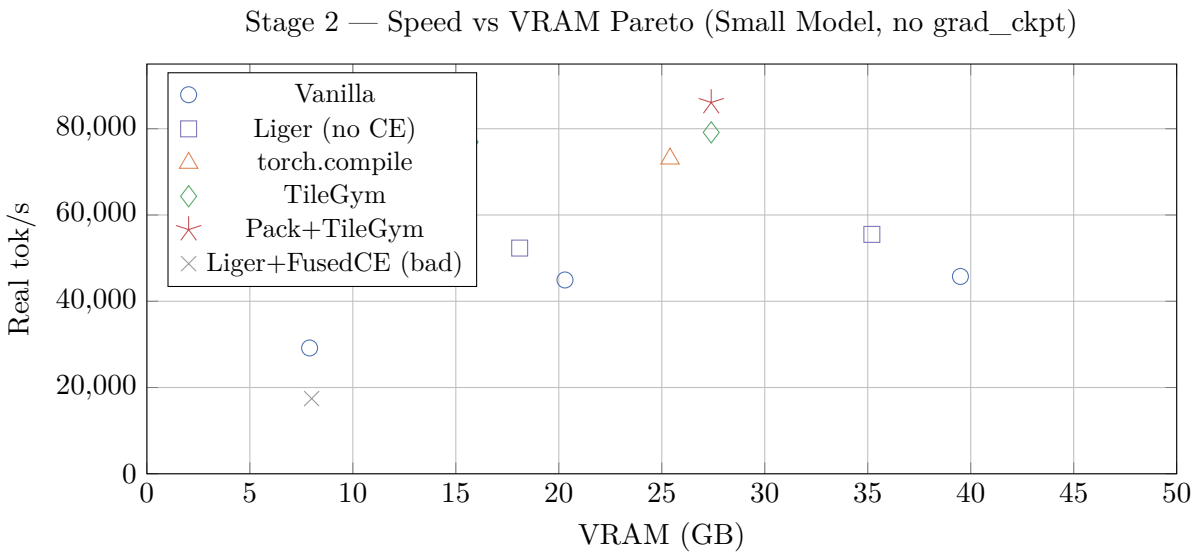

Figure 2: Stage 2 Pareto frontier. TileGym dominates again, but torch.compile is a close second without external dependencies. Liger with FusedCE (× marker) is clearly below the frontier—a cautionary example of blind kernel application.

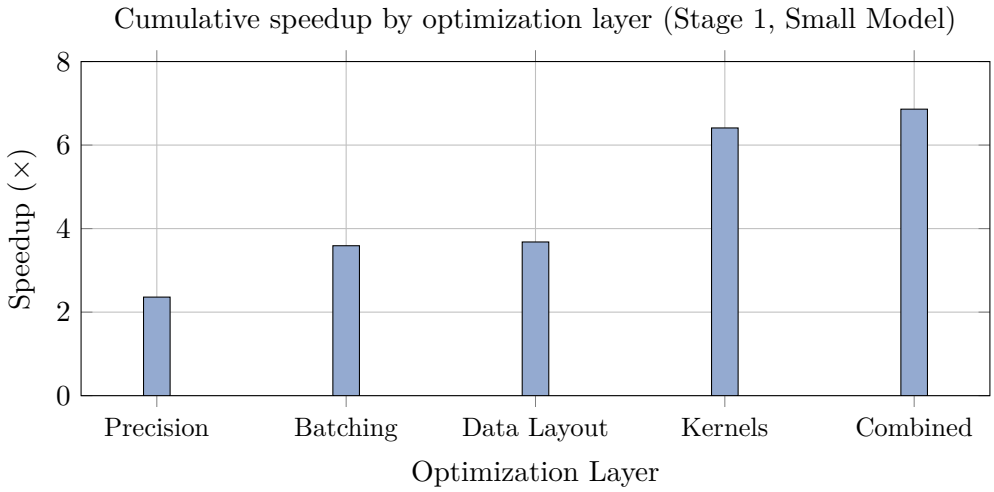

Figure 3: Each layer contributes multiplicatively. Precision is the highest-ROI single change. Kernel fusion provides the second-largest jump. Data layout (bucketing/packing) provides the final margin.

11 Discussion

Generalizability. Our framework applies to any VLM following the two-stage paradigm. The specific kernel choice (TileGym vs torch.compile) depends on model architecture, but the five-layer approach and stage-dependent configuration principle are universal. We expect these findings to transfer to InternVL, Qwen-VL, LLaVA-NeXT, and similar architectures.

When is this guide most valuable?

- Researchers training VLMs on a single high-end GPU (A100, H100, Blackwell)
- Teams establishing per-device baselines before scaling to distributed training
- Practitioners evaluating kernel libraries (Liger, TileGym, Unsloth) for their models
- Anyone encountering unexplained slowdowns after applying "optimization" libraries

Limitations. (1) We test on one GPU architecture (Blackwell); relative kernel performance may differ on Ampere/Hopper. (2) We use one dataset (sharegpt4v/coco); extreme length distributions may shift the packing vs bucketing tradeoff. (3) We do not evaluate quantized training (QLoRA, FP8) which may shift the Pareto frontier further.

Future Work. Multi-GPU scaling experiments (FSDP, tensor parallelism) using our optimized per-device configuration as the starting point. FP8 training on Blackwell. Automated configuration search using the five-layer framework as the search space.

12 Conclusion

We presented a systematic, reproducible guide for optimizing single-GPU VLM training, achieving 6.86× (Stage 1) and 2.95× (Stage 2) speedup through five optimization layers. Our central finding is that **optimization must be stage-aware**: the same technique (FusedCE, gradient checkpointing) can be beneficial in one context and catastrophic in another.

The key lessons for practitioners are:

1. Precision fixes yield the highest return on effort (2–3× for 2 lines of code).
2. Kernel libraries require per-stage configuration—never apply them blindly.
3. Gradient checkpointing is a premature optimization on high-VRAM hardware.
4. Verify kernel compatibility with your model's specific dimensions before committing.
5. Batch size has a saturation point; beyond it, you only waste VRAM.

All code, benchmark scripts, and raw experiment traces are available at https://github.com/duoan/SiQ_VL.

References

[1] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[2] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. In *CVPR*, 2024.

[3] Z. Chen, J. Wu, W. Wang, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *CVPR*, 2024.

[4] J. Bai, S. Bai, S. Yang, et al. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv:2308.12966*, 2023.

[5] P. Wang, S. Bai, S. Tan, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv:2409.12191*, 2024.

[6] H. Liu, C. Li, Y. Li, B. Li, et al. LLaVA-NeXT: Improved reasoning, OCR, and world knowledge. https://llava-vl.github.io/blog/2024-01-30-llava-next/, 2024.

[7] L. Beyer, A. Steiner, A. S. Pinto, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv:2407.07726*, 2024.

[8] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *ICCV*, 2023.

[9] A. Yang, B. Yang, B. Hui, et al. Qwen2.5 technical report. *arXiv:2412.15115*, 2024.

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[11] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[12] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL Workshop*, 2019.

[13] P. Hsu, Y. Shi, Y. Zhu, et al. Liger-Kernel: Efficient Triton kernels for LLM training. *arXiv:2410.10989*, 2024.

[14] D. Han and M. Han. Unsloth: 2x faster LLM fine-tuning with 80% less memory. https://github.com/unslothai/unsloth, 2024.

[15] T. He, et al. FlexAttention: A programming model for generating optimized attention kernels. *PyTorch Blog*, 2024.

[16] J. Ansel, E. Yang, H. He, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ASPLOS*, 2024.

[17] M. M. Krell, N. Kosec, S. P. Perez, and A. Fitzgibbon. Efficient sequence packing without cross-contamination. *arXiv:2107.02027*, 2021.

[18] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.

[19] E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[20] M. Shoeybi, M. Patwary, R. Puri, et al. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*, 2019.

[21] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *KDD*, 2020.

[22] A. Chowdhery, S. Narang, J. Devlin, et al. PaLM: Scaling language modeling with Pathways. *JMLR*, 24:1–113, 2023.

[23] P. Micikevicius, S. Narang, J. Alben, et al. Mixed precision training. In *ICLR*, 2018.